

LegalEase

AI-Powered Legal Document Generator

Complete Project Solution: Architecture, Source Code and Run Guide

FastAPI + Streamlit + Google Gemini + python-docx + FPDF

1. Overview

LegalEase takes four user inputs (document type, parties, terms, effective date), sends them to a FastAPI backend, which builds a structured prompt for Gemini and returns the drafted legal text. The Streamlit frontend shows a styled preview, lets the user edit the text, and exports it as .txt, .docx (logo, Times New Roman, terms table, footer) or .pdf (logo header and footer on every page).

2. Architecture

Layer	Technology	File
Frontend	Streamlit	frontend/app.py
Backend	FastAPI + Uvicorn	legalEaseAPI/main.py, routes.py
AI Core	Gemini (google-generativeai)	ai_core/gemini_generator.py
Formatters	python-docx, FPDF	ai_core/generator.py
Config	python-dotenv	.env, config.py

Flow: User Interface (Streamlit) → POST /generate (FastAPI) → GeminiDocumentGenerator → Gemini API → generated text → preview / edit / export (.txt, .docx, .pdf).

3. Folder Structure

```
LEGALEASE/  
|-- ai_core/  
|   |-- gemini_generator.py  
|   |-- generator.py          (sanitize, docx, pdf, html preview)  
|-- frontend/  
|   |-- app.py  
|-- Image/  
|   |-- Logo.png              (used in DOCX/PDF)  
|   |-- inverseLogo.png       (used in web UI)  
|-- legalEaseAPI/  
|   |-- main.py  
|   |-- routes.py  
|-- .env  
|-- config.py  
|-- requirements.txt  
|-- run.sh
```

Note: create empty `__init__.py` files inside `ai_core/` and `legalEaseAPI/` so the imports resolve reliably. Place your own logo images in the `Image/` folder (the code skips the logo gracefully if missing).

4. Setup (Milestones 1 and 5)

```
python -m venv venv  
venv\Scripts\activate          # Windows   (Linux/Mac: source venv/bin/activate)  
pip install -r requirements.txt  
# put your key in .env -> GEMINI_API_KEY=... (get one at aistudio.google.com)
```

Model note: the project brief names `gemini-1.5-pro`, but Google has retired the 1.5 models, so the code reads the model name from `GEMINI_MODEL` in `.env` and defaults to **gemini-2.5-flash**. You can set it to `gemini-2.5-pro` for higher quality. Check Google's model list if a name is rejected.

5. Source Code

requirements.txt

```
fastapi
uvicorn
streamlit
python-docx
fpdf
Pillow
requests
google-generativeai
python-dotenv
```

.env

```
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL=gemini-2.5-flash
```

config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
BACKEND_URL = os.getenv("BACKEND_URL", "http://localhost:8000")

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
WEB_LOGO_PATH = os.path.join(BASE_DIR, "Image", "inverseLogo.png")
DOC_LOGO_PATH = os.path.join(BASE_DIR, "Image", "Logo.png")
```

ai_core/gemini_generator.py

```
import google.generativeai as genai
from config import GEMINI_API_KEY, GEMINI_MODEL

genai.configure(api_key=GEMINI_API_KEY)

class GeminiDocumentGenerator:
    def __init__(self, model_name: str = GEMINI_MODEL):
        self.model = genai.GenerativeModel(model_name)

    def generate_document(self, document_type, parties, terms, dates):
        term_list = [t.strip() for t in terms.split(";") if t.strip()]
        terms_block = "\n".join(f"- {t}" for t in term_list)

        prompt = (
            f"Generate a comprehensive legal document titled "
            f"'{document_type}'.\n"
            f"Involved parties: {parties}\n"
            f"Effective Date: {dates}\n"
            f"Terms and conditions:\n{terms_block}\n"
            "Ensure formal legal structure with numbered sections and "
            "legal clauses (definitions, obligations, payment, "
            "confidentiality, termination, governing law, "
            "severability, signatures). Use plain text headings "
            "such as '1. Services:' and avoid markdown symbols."
        )
        response = self.model.generate_content(prompt)
        return response.text
```

legalEaseAPI/main.py

```
from fastapi import FastAPI
from legalEaseAPI.routes import router

app = FastAPI(title="LegalEase - AI Legal Document Generator")

# Include API routes
app.include_router(router)

@app.get("/")
def home():
    return {"message": "Welcome to LegalEase AI Legal Document "
                      "Generator API"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

legalEaseAPI/routes.py

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
gemini_generator = GeminiDocumentGenerator()

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str

@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    try:
        response = gemini_generator.generate_document(
            request.document_type,
            request.parties,
            request.terms,
            request.dates,
        )
        return {"document": response}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

ai_core/generator.py

```
import html
import io
import re

from docx import Document
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.shared import Inches, Pt
from fpdf import FPDF

from config import DOC_LOGO_PATH

FOOTER_TEXT = "LegalEase Inc. | contact@legalease.com | All Rights Reserved."
HEADING_RE = re.compile(r"^\s*(\d+\.\s+.+:[A-Z][A-Z ,.&-]{3,}:?)\s*$")

def sanitize_text(text: str) -> str:
    """Remove markdown symbols and typographic characters."""
    replacements = {
        "\u2018": "'", "\u2019": "'", "\u201c": '"', "\u201d": '"',
        "\u2013": "-", "\u2014": "-", "\u2022": "-", "\u2026": "...",
        "\u00a0": " ",
    }
    for old, new in replacements.items():
        text = text.replace(old, new)
    text = re.sub(r"``*\*|_|`", "", text) # bold / code marks
    text = re.sub(r"#{1,6}\s*", "", text, flags=re.M) # markdown headings
    return text.encode("latin-1", "ignore").decode("latin-1").strip()

def _is_heading(line: str) -> bool:
    return bool(HEADING_RE.match(line))

# ----- DOCX
def format_docx(text: str, doc_type: str, terms: str = "") -> bytes:
    doc = Document()
    style = doc.styles["Normal"]
    style.font.name = "Times New Roman"
    style.font.size = Pt(12)

    try:
        doc.add_picture(DOC_LOGO_PATH, width=Inches(2))
        doc.paragraphs[-1].alignment = WD_ALIGN_PARAGRAPH.CENTER
    except Exception:
        pass # logo is optional

    title = doc.add_heading(doc_type, level=1)
    title.alignment = WD_ALIGN_PARAGRAPH.CENTER

    for line in sanitize_text(text).split("\n"):
        line = line.strip()
        if not line:
            continue
        p = doc.add_paragraph()
        run = p.add_run(line)
        if _is_heading(line):
            run.bold = True

    # Auto-generated Terms table from semicolon-separated input
    term_list = [t.strip() for t in terms.split(";") if t.strip()]
    if term_list:
        doc.add_heading("Summary of Terms", level=2)
        table = doc.add_table(rows=1, cols=2)
        table.style = "Table Grid"
        table.rows[0].cells[0].text = "No."
        table.rows[0].cells[1].text = "Term"
        for i, t in enumerate(term_list, 1):
            row = table.add_row().cells
            row[0].text = str(i)
            row[1].text = t

    footer = doc.sections[0].footer.paragraphs[0]
    footer.text = FOOTER_TEXT
    footer.alignment = WD_ALIGN_PARAGRAPH.CENTER

    buf = io.BytesIO()
    doc.save(buf)
```

```

return buf.getvalue()

# ----- PDF
class LegalPDF(FPDF):
    def __init__(self, doc_type):
        super().__init__()
        self.doc_type = doc_type

    def header(self):
        try:
            self.image(DOC_LOGO_PATH, x=85, y=8, w=40)
        except Exception:
            pass
        self.ln(22)
        self.set_font("Arial", "B", 13)
        self.cell(0, 8, self.doc_type, ln=True, align="C")
        self.ln(4)

    def footer(self):
        self.set_y(-15)
        self.set_font("Arial", "I", 8)
        self.cell(0, 10, FOOTER_TEXT, align="C")

def format_pdf(text: str, doc_type: str) -> bytes:
    pdf = LegalPDF(sanitize_text(doc_type))
    pdf.set_auto_page_break(auto=True, margin=20)
    pdf.add_page()

    for line in sanitize_text(text).split("\n"):
        line = line.strip()
        if not line:
            pdf.ln(3)
            continue
        if _is_heading(line):
            pdf.set_font("Arial", "B", 11)
        else:
            pdf.set_font("Arial", "", 11)
        if line.startswith("- "):
            line = " - " + line[2:]          # bullet-style terms
        pdf.multi_cell(0, 6, line)
        pdf.ln(1)

    out = pdf.output(dest="S")
    return out.encode("latin-1") if isinstance(out, str) else bytes(out)

# ----- HTML
def format_html_preview(text: str) -> str:
    parts = []
    for line in sanitize_text(text).split("\n"):
        line = html.escape(line.strip())
        if not line:
            continue
        if _is_heading(line):
            parts.append(f"<h4 style='margin:14px 0 4px'>{line}</h4>")
        elif line.startswith("- "):
            parts.append(f"<li>{line[2:]}</li>")
        else:
            parts.append(f"<p style='margin:4px 0'>{line}</p>")
    return "".join(parts)

```

frontend/app.py

```
import os
import sys

import requests
import streamlit as st

sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from ai_core.generator import (format_docx, format_html_preview,
                                format_pdf, sanitize_text)
from config import BACKEND_URL, WEB_LOGO_PATH

st.set_page_config(page_title="LegalEase", layout="centered")

# ---- Logo + title -----
col1, col2, col3 = st.columns([1, 2, 1])
with col2:
    if os.path.exists(WEB_LOGO_PATH):
        st.image(WEB_LOGO_PATH, use_container_width=True)
    st.markdown("<h2 style='text-align: center;'>AI Legal Document "
                "Generator</h2>", unsafe_allow_html=True)

# ---- Inputs -----
document_type = st.text_input("Document Type (Ex: Agreement, Contract, NDA)")
parties = st.text_area("Parties Involved")
terms = st.text_area("Terms & Conditions (Use semicolons for bullet points)")
dates = st.text_input("Effective Date")

# ---- Generate -----
if st.button("Generate Document"):
    if not all([document_type.strip(), parties.strip(),
                terms.strip(), dates.strip()]):
        st.warning("Please fill in all fields.")
    else:
        with st.spinner("Generating document..."):
            try:
                response = requests.post(
                    f"{BACKEND_URL}/generate",
                    json={"document_type": document_type,
                        "parties": parties,
                        "terms": terms,
                        "dates": dates},
                    timeout=120,
                )
                response.raise_for_status()
                st.session_state.generated_text = sanitize_text(
                    response.json()["document"])
                st.session_state.doc_type = document_type
                st.session_state.terms = terms
                st.session_state.show_edit = False
                st.success("Document Generated Successfully!")
            except Exception as e:
                st.error(f"Error: {e}")
else:
    if "generated_text" not in st.session_state:
        st.info("Click 'Generate Document' to start")

# ---- Preview / Edit / Download -----
if "generated_text" in st.session_state:
    styled_html = format_html_preview(st.session_state.generated_text)
    st.markdown(
        "<div style='background:#111827;color:#e5e7eb;padding:16px;"
        "border-radius:10px;height:350px;overflow-y:auto;'>"
        f"{styled_html}</div>",
        unsafe_allow_html=True,
    )

    if st.button("Click to Edit Document"):
        st.session_state.show_edit = True

    if st.session_state.get("show_edit"):
        edited = st.text_area("Edit Document Below:",
                               st.session_state.generated_text, height=300)
        st.session_state.generated_text = edited

text = st.session_state.generated_text
dtype = st.session_state.get("doc_type", "Legal Document")
```

```

fname = dtype.replace(" ", "_").lower()

st.download_button("Download as .TXT", data=text,
                   file_name=f"{fname}.txt", mime="text/plain")
st.download_button(
    "Download as .DOCX",
    data=format_docx(text, dtype, st.session_state.get("terms", "")),
    file_name=f"{fname}.docx",
    mime="application/vnd.openxmlformats-officedocument."
        "wordprocessingml.document")
st.download_button("Download as .PDF", data=format_pdf(text, dtype),
                   file_name=f"{fname}.pdf", mime="application/pdf")

```

run.sh

```

#!/bin/bash
# Start backend and frontend
uvicorn legalEaseAPI.main:app --reload &
streamlit run frontend/app.py

```

6. Running the Application (Milestone 5)

```

# Terminal 1 - backend
uvicorn legalEaseAPI.main:app --reload           # http://127.0.0.1:8000

# Terminal 2 - frontend
streamlit run frontend/app.py                   # http://localhost:8501

```

7. Test Input

```

Document Type : Freelance Work Contract
Parties       : Jane Doe (Service Provider), TechNova Inc. (Client)
Terms         : Work must be delivered by May 15, 2025;
               Payment will be made within 7 days of invoice;
               The client retains intellectual property rights;
               Confidentiality must be maintained at all times
Effective Date: April 15, 2025

```

Verify: preview renders, the Edit box changes the exports, and the .docx and .pdf show the logo, terms and footer.

8. Deployment Options (Activity 5)

Backend: Render, Railway, Fly.io or a VPS (start command: `uvicorn legalEaseAPI.main:app --host 0.0.0.0 --port $PORT`). Frontend: Streamlit Community Cloud, with `BACKEND_URL` set to the deployed API address and `GEMINI_API_KEY` stored as a secret, never committed to Git.

9. Disclaimer

AI-generated documents are drafts and not a substitute for advice from a qualified lawyer. Placeholders such as [State] and [Address] must be reviewed and completed before use.